МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ З ЛАБОЛАТОРНОЇ

№3

на тему **“Нездатні до змін структури даних (immutability) у Python ”**

Варіант 15

Виконала:

Студентка 2 курсу

групи ФЕП-22

Сулятицька Христина

Перевірив:

Аспірант

Остріков Данило Андрійович

Львів-2026

Мета роботи. Ознайомитися з незмінними структурами даних у Python та навчитися застосовувати tuple, frozenset, dataclasses із параметрами frozen=True і slots=True та MappingProxyType. Реалізувати оновлення стану без мутації початкових об’єктів, створити тести на незмінність і хешованість, а також порівняти час виконання операцій для мутабельної та незмінної моделей.

Теоретичні відомості:

Незмінні структури даних не дозволяють змінювати свій вміст після створення. Такий підхід зменшує кількість прихованих побічних ефектів, спрощує тестування та робить поведінку програми передбачуванішою. Замість зміни наявного об’єкта створюється новий об’єкт із потрібними значеннями.

У Python для реалізації незмінності можна використовувати tuple замість list, frozenset замість set, @dataclass(frozen=True, slots=True) для доменних сутностей і MappingProxyType для словників, доступних лише для читання. MappingProxyType є незмінним поданням словника, тому перед його створенням необхідно скопіювати початкові метадані.

Для оновлення frozen dataclass застосовується dataclasses.replace(). Функція отримує початковий об’єкт і повертає його копію зі зміненими полями. Такий спосіб відповідає принципу pure update: початковий об’єкт залишається без змін.

Хід роботи

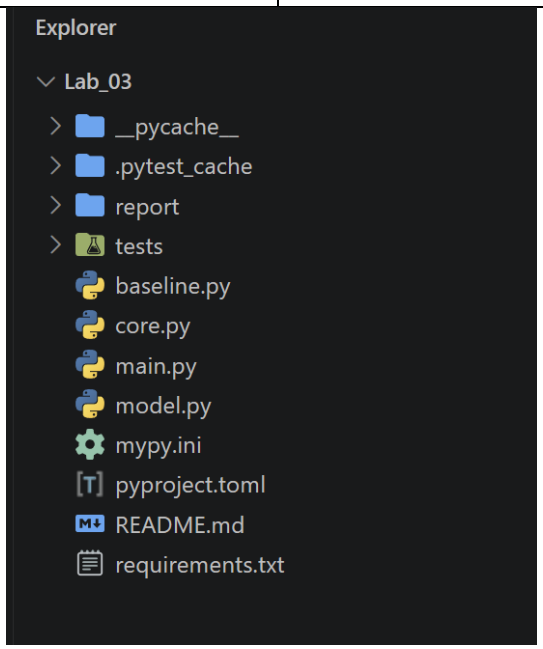
Для 15-го варіанта реалізовано систему задач проєкту. Основною сутністю є задача Task, яка містить ідентифікатор, назву, опис, статус, пріоритет, коментарі, мітки та метадані. Для зберігання коментарів використовується незмінний кортеж, для міток — незмінна множина, а метадані доступні лише для читання.

1. Створення структури проєкту

Проєкт розділено на модулі відповідно до їх призначення. У model.py описано доменні сутності Task і Comment та функцію створення read-only метаданих. У core.py розміщено чисті функції оновлення. Файл baseline.py містить мутабельну модель, яка використовується лише для порівняння продуктивності. У main.py реалізовано консольне меню. Автоматизовані перевірки розміщено в каталозі tests.

Файл	**Призначення**

model.py	Незмінні класи Task і Comment, створення MappingProxyType.
core.py	Pure update-функції та бізнес-операції над задачами.
baseline.py	Мутабельна модель для порівняльного вимірювання.
main.py	Інтерактивне консольне меню.
tests/test_immutability.py	Перевірки незмінності, метаданих, хешованості та оновлень.
tests/test_performance.py	Порівняння часу мутабельних і незмінних операцій.
requirements.txt	Залежності для тестування та перевірки якості коду.
pyproject.toml, mypy.ini	Налаштування інструментів перевірки проєкту.



2. Реалізація незмінної моделі у файлі model.py

Для опису коментаря створено dataclass Comment з полями id, author, text і created_at. Клас Task містить дані задачі та використовує frozen=True і slots=True. Параметр frozen=True забороняє змінювати поля після створення об’єкта, а slots=True обмежує створення довільних атрибутів.

Поле comments має тип tuple[Comment, ...], labels — frozenset[str], a metadata — Mapping[str, object]. У методі __post_init__ вхідні колекції нормалізуються до незмінних типів. Метадані копіюються та обгортаються в MappingProxyType, тому зміни початкового словника не впливають на створений об’єкт.

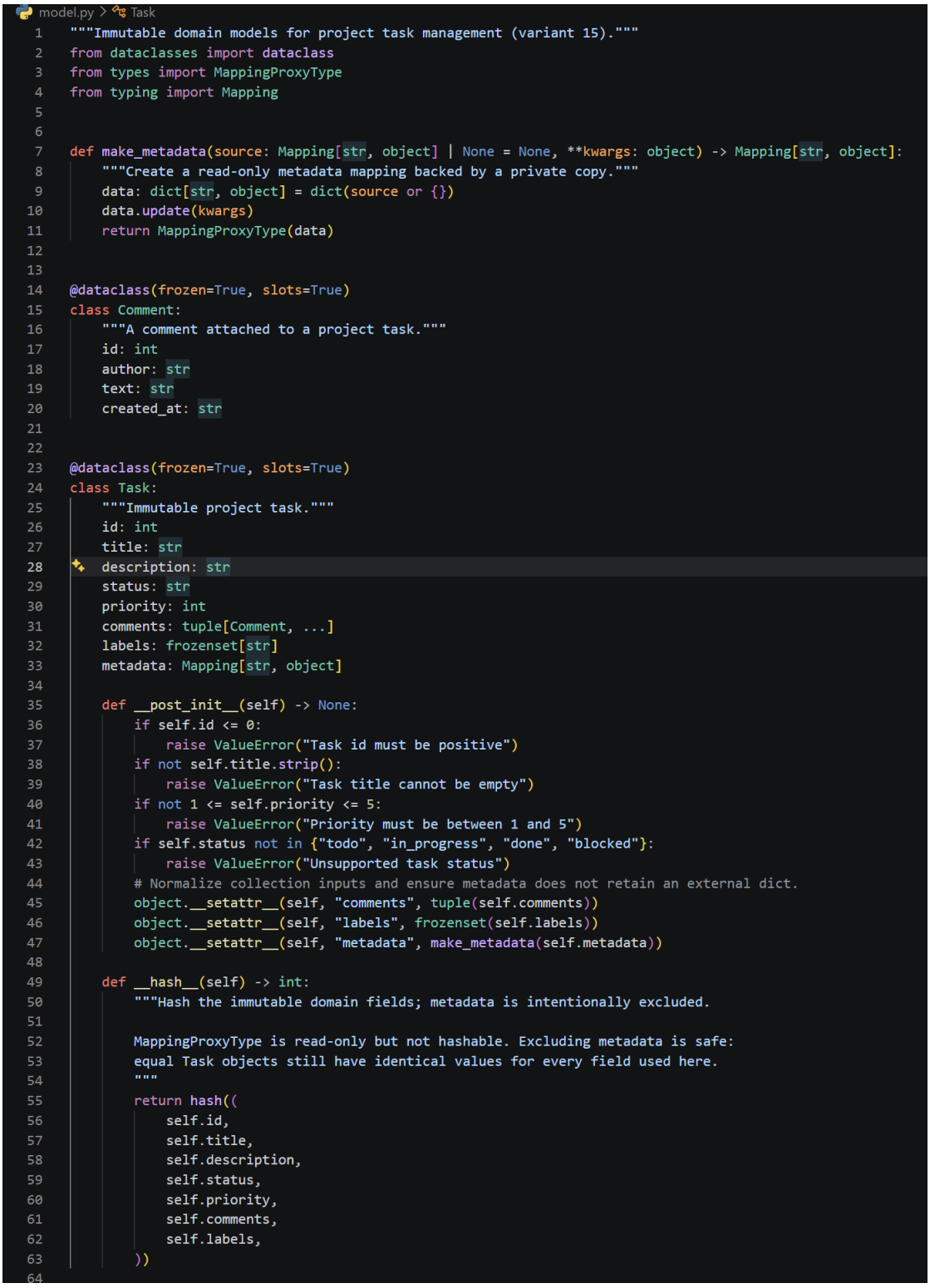


Рисунок 1 — Оголошення незмінних доменних сутностей

Для Task також реалізовано власний метод __hash__. MappingProxyType не підтримує хешування, тому метадані не включаються до обчислення хешу. Хеш формується за незмінними доменними полями: ідентифікатором, назвою, описом, статусом, пріоритетом, коментарями та мітками. Це дає змогу використовувати об’єкт Task як ключ словника або елемент множини.

3. Реалізація чистих функцій оновлення у файлі core.py

У модулі core.py реалізовано чотири основні операції відповідно до індивідуального завдання: додавання коментаря, додавання мітки, зміна статусу та зміна пріоритету. Кожна функція повертає новий об’єкт Task і не змінює початкову задачу.



Рисунок 2 — Приклад pure update-функцій

Під час додавання коментаря формується новий кортеж шляхом конкатенації. Перед додаванням перевіряється унікальність ідентифікатора коментаря. Для додавання мітки використовується операція об’єднання frozenset. Зміна статусу та пріоритету виконується через dataclasses.replace(). У функціях передбачено перевірку допустимих значень статусу, пріоритету та непорожності мітки.

4. Реалізація консольної оболонки у файлі main.py

У файлі main.py реалізовано інтерактивне консольне меню. Після запуску створюється початкова задача, після чого користувач може переглянути її стан, додати коментар, додати мітку, змінити статус або змінити пріоритет. Для завершення роботи передбачено окремий пункт меню.

Після кожної успішної операції змінна task отримує нову версію задачі, повернену відповідною pure-функцією. Попередній об’єкт при цьому не змінюється. Некоректні значення обробляються через винятки ValueError і TypeError, після чого програма продовжує роботу.

5. Створення та виконання тестів

Для перевірки реалізації створено автоматизовані тести pytest. Вони перевіряють заборону зміни полів frozen dataclass, використання tuple і frozenset, read-only поведінку MappingProxyType, копіювання початкових метаданих, хешованість Task та правильність pure update-функцій.

- зміна полів Task і Comment викликає FrozenInstanceError;
- коментарі, мітки та метадані мають незмінні типи;
- додавання коментаря або мітки повертає новий об’єкт, не змінюючи початкову задачу;
- зміна статусу та пріоритету не впливає на початкові значення;
- зміна початкового словника метаданих не змінює MappingProxyType;
- Task можна використовувати як елемент множини та ключ словника;
- повторне використання ID коментаря та некоректні значення відхиляються.

Під час контрольного запуску команди pytest -q --durations=5 -s усі 12 тестів завершилися успішно.

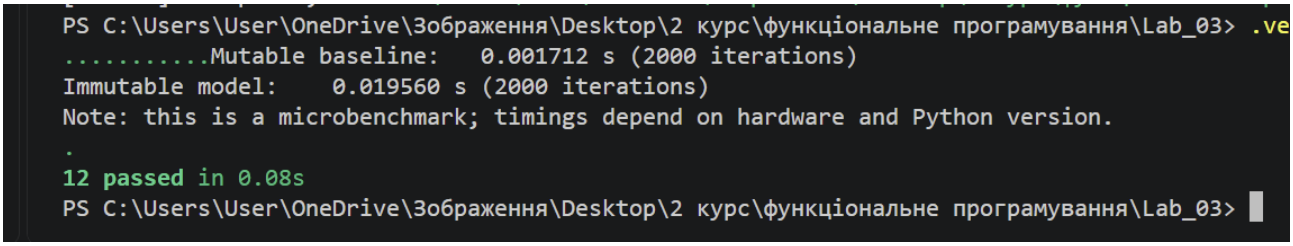


Рисунок 3 — Результат виконання тестів

6. Порівняння продуктивності

Для оцінювання накладних витрат незмінної моделі створено tests/test_performance.py. У тесті порівнюється виконання двох сценаріїв: оновлення мутабельного об’єкта MutableTask та створення нових версій immutable-об’єкта Task. Для вимірювання використовується timeit, кількість повторень становить 2000.

Реалізація	**Час виконання**
Мутабельна модель	0,001353 с
Незмінна модель	0,019642 с

За результатами одного контрольного запуску мутабельна реалізація виконала операції швидше. Незмінний підхід має додаткові витрати, оскільки під час оновлення створюється новий об’єкт і формується нове значення колекцій. Наведені значення є результатом конкретного запуску та можуть відрізнятися залежно від обладнання й версії Python. Водночас незмінність забезпечує кращу передбачуваність стану та спрощує перевірку відсутності побічних ефектів.

7. Перевірка стилю та типів

У проєкті додано requirements.txt із pytest, mypy, black і ruff. У файлах pyproject.toml та mypy.ini задано налаштування інструментів. Для локальної перевірки після встановлення залежностей передбачено команди:

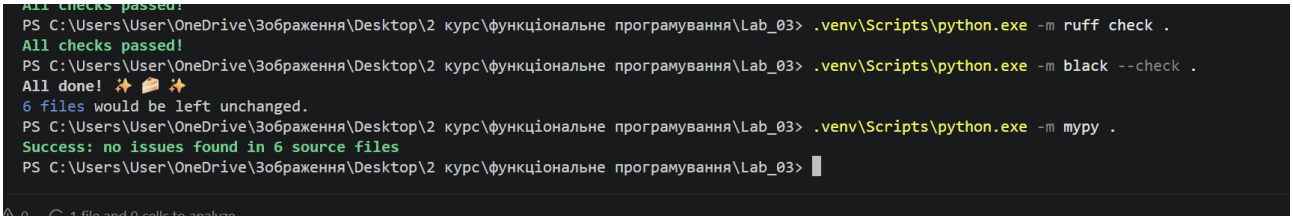


Рисунок 4 — Команди перевірки проєкту

```
mypy .
black --check .
ruff check .
pytest -q --durations=5 -s
```

Під час підготовки звіту підтверджено успішне проходження 12 тестів pytest. Перевірки mypy, Black і Ruff необхідно виконати в локальному середовищі після встановлення відповідних пакетів; результат їх виконання в цьому звіті не позначається як підтверджений.

Висновок:

У ході лабораторної роботи я реалізувала систему задач проєкту відповідно до 15-го варіанта. Для предметної моделі створила незмінні сутності Task і Comment на основі dataclass із параметрами frozen=True та slots=True. Коментарі зберігаються в tuple, мітки — у frozenset, а метадані захищені за допомогою MappingProxyType із попереднім копіюванням початкового словника.

Я реалізувала чисті функції додавання коментаря та мітки, зміни статусу й пріоритету. Кожна операція повертає нову версію задачі, не змінюючи початковий об’єкт. Також додала консольне меню, перевірки коректності даних і тести pytest. Усі 12 тестів пройшли успішно. Окремо я порівняла час виконання мутабельної та незмінної моделей і побачила, що незмінний підхід має більші витрати часу через створення нових об’єктів, проте забезпечує контрольований стан даних і спрощує тестування.

Таким чином, я закріпила використання незмінних структур даних у Python та навчилася реалізовувати оновлення без мутації початкових об’єктів.